

2026-09-18

Arity: A Standalone Language Server for R Package Development

R Consortium ISC proposal

Johan Larsson 

Department of Mathematical Sciences, University of Copenhagen

Executive Summary

R package development involves working across source files, dependencies, and documentation. I created [Arity](#) to help developers navigate and check this code from their editor without running R. Arity is an MIT-licensed language server written in Rust, with an integrated formatter and linter. It supports VS Code, Positron, Neovim, Zed, and other editors through the Language Server Protocol (LSP).

This project will improve Arity's handling of shared documentation and function names, and provide function signatures and help for packages that are not installed locally. I will test these improvements in complete editing workflows and publish the results. I ask for USD 10,000 to fund 125 hours of labor.

Signatories

Project team

I am Johan Larsson, Arity's maintainer and a postdoctoral researcher in statistics and machine learning at the University of Copenhagen's Department of Mathematical Sciences. I also develop and maintain several R packages, including [eulerr](#), [SLOPE](#), and [qualpalr](#).

Contributors

I will carry out the work myself. The budget does not include development time from other contributors.

Consulted

I'm grateful for feedback from Tatsuya Shima (maintainer of [arf](#), [rd2qmd](#), [r-documentation-rs](#), and [mini-roxygen](#)), whose suggestions helped clarify and improve the proposal.

The Problem

Package authors need to follow references across files, check imported functions, and keep documentation consistent with their code. Language servers help with these tasks through completion, hover information, signature help, navigation, symbol renaming, and call hierarchy. These features help developers catch errors while editing and make changes across a package.

There are currently three language servers for R: [languageserver](#), [Ark](#), and Arity. [languageserver](#) is a mature R package that runs in an R session and supports many editors. Ark is a newer tool from Posit, written in Rust and built around its own R kernel and Positron. Arity is faster and uses less memory than [languageserver](#), and it does not require Positron to run.

Arity still has limitations that affect package development. It can misinterpret shared roxygen documentation, miss undocumented parameters, and handle references and call hierarchy inconsistently. Renaming can also produce invalid R syntax, for example when a function is used as an infix operator.

Arity also needs better support for dependencies that are not installed locally. It already provides function signatures and help for installed packages, but for other packages it has access only to exported names. Developers therefore need to install their dependencies to get signatures and help in the editor. Richer metadata would remove this requirement. The editor and command-line interface (CLI) also need to use the same cached metadata when downloads are disabled.

These improvements will help package authors and contributors catch errors before they reach users. Publishing the metadata separately will also let other R tools use it, while the tests and reported limitations will help their developers evaluate similar behavior.

The Proposal

Overview

I will improve Arity's analysis of package code, extend its metadata for dependencies, and test these changes together in complete editing workflows.

Detail

Minimum Viable Product

1. **Language-server reliability.** I will improve Arity's handling of shared and inherited roxygen documentation. References, renaming, and call hierarchy will handle quoted function names consistently. Renaming will produce valid R syntax, including in infix calls, or Arity will refuse the operation.
2. **Package knowledge for editor assistance.** I will publish versioned indexes for at least 100 CRAN packages meeting the quality criteria below. These will include packages outside the existing bundle and dependencies used in the evaluation workflows. The indexes will contain exported names, symbol kinds, recoverable function arguments and defaults, and help. Arity

will use these indexes to provide completion, hover information, and signature help without requiring local package installation.

I will document the schema, automate weekly updates, and add a prefetch command for downloading indexes in bulk. The editor and CLI will use the same cached metadata and work with downloads disabled. Network access will remain an explicit user choice. Each index will record the package version and how the metadata was extracted.

3. **Validation and adoption.** I will test complete editing workflows on ten packages with cross-file references, imports, shared documentation, and nonstandard evaluation. These tests will also compare diagnostics from the editor and CLI. I will update the guides for VS Code, Positron, Neovim, and continuous integration (CI), explaining how to use Arity alongside other R tools and how to work offline.

Each behavior change will have regression tests and tests for cases that should remain unchanged. I will build on Arity's existing corpus checks, protocol tests, and benchmarks.

Architecture

Arity analyzes code across a project and updates its analysis as files change. Its architecture for parsing, formatting, and language intelligence is inspired by [rust-analyzer](#). The separately published [arity-parser](#) and [arity-formatter](#) libraries are already used by [mini-roxygen-core](#), which generates R documentation and NAMESPACE files from roxygen comments.

Arity's metadata harvester already reads serialized R objects and installed-package help without running R. Its binary parsing and Rd decoding overlap with [r-documentation-rs](#). I will coordinate with its maintainer to consolidate these readers in [r-documentation-rs](#), reducing duplication and making improvements available to other R tools. Arity will continue to handle package discovery, export resolution, metadata indexing, caching, and editor integration.

As of September 2026, Arity bundles export names for over 500 packages and has a workflow to refresh them weekly. The remote client already supports downloads, disk caching, and versioned indexes, but accepts only exported names. I will add a pipeline to publish richer indexes, extend the client to supply signatures and help, and make cached exports available to the CLI. Local indexes will take precedence, and Arity will show the package version when it uses a remote index.

Assumptions

Static analysis cannot determine what every name refers to at runtime. Arity will refuse uncertain rewrites, and I will document unsupported cases. Remote indexes describe current releases, which may differ from the versions a project uses.

The main risks are incorrect or incomplete metadata, confusion between package versions, and errors when renaming quoted function names. I will record missing fields and failed extractions, and older metadata will retain the version of the package it describes. I will keep the work within budget by selecting packages and using existing infrastructure. Historical package versions and signatures that require running R are outside the scope of this project.

External Dependencies

For the pilot, I will extract metadata from unpacked [R-universe binary packages](#), falling back on packages installed in CI when needed. I will check the complete export lists and samples of signatures and help against the same package versions in R, and use [roxygen2](#) to check documentation behavior. These checks will use R, but users will not need a running R session or installed dependencies to use

the resulting metadata. The source packages and extracted data will retain their version and license information.

I am responsible for delivering the project, and the budget does not assume development time from other maintainers. The work does not require changes to R.

Project Plan

Start-Up Phase

The first milestone (M1) will begin in January 2027. I will pin versions of ten evaluation packages, define their editing workflows, and record baseline diagnostics and response times. I will reproduce the analysis problems described above and document the setup so that I can compare the final results with this baseline. Existing reproductions from *future* provide a starting point.

Initial extraction checks in September 2026 found the required fields for every exported function in glue, withr, R6, and cli. For magrittr, coverage was 16 of 42 functions, largely because primitive aliases lack formal arguments. The checks also found S3 methods incorrectly listed as exports. The metadata pilot will build on these results. It will use glue, withr, R6, and cli, along with tinytable to test a package outside the existing bundle. I will also use magrittr to test incomplete indexes. I will fix the S3 export errors during M1, before publication.

During M1, I will identify the overlapping code, missing APIs, and tests needed to consolidate the readers in r-documentation-rs. I will use the pilot packages to compare arguments, defaults, and help with results from both the existing extractor and R, and estimate the work for M2. If full consolidation would exceed the available hours, I will retain the parts of Arity's extractor needed to complete the project and defer the remaining consolidation work.

Before implementation begins in M2, five pilot packages must pass the quality criteria below. Arity must also provide signatures and help without local installation for packages both inside and outside the existing bundle. I will estimate the cost of weekly updates. If the pilot cannot meet these requirements within budget, I will agree on a reduced scope with the ISC, giving priority to the metadata features and offline access.

Technical Delivery

Milestone	Target date	Deliverables	Hours	Cost (USD)
M1: Baseline and pilot	January 31, 2027	Baseline, metadata prototype, consolidation plan, quality checks, regression cases	30	2,400
M2: Implementation	February 28, 2027	Reliability fixes, rich metadata publication and integration, prefetch, shared offline cache	65	5,200
M3: Validation and release	March 31, 2027	Workflow validation, annotated comparison, guides, release, report, maintenance instructions	30	2,400
Total			125	10,000

M1 includes planning and estimating the consolidation work. Changes to the shared readers and their integration into Arity must fit within M2's 65 hours. The estimates include regression tests, with eight hours in M2 set aside for unexpected work. The schedule assumes a January start after agreement. The ISC and I will agree on any changes to the dates before work begins. I will track progress and completion criteria in public GitHub issues.

Other Aspects

Code and tests will use Arity's MIT license. Evaluation sources and examples will retain applicable notices. I will publish the schema, code for the publishing pipeline, documentation, and results, along with instructions for rebuilding or mirroring the dataset.

Furthermore, I will invite package authors and tooling developers to try a prerelease and will also write R Consortium blog posts to announce the project and share results, examples, and limitations.

Budget & Funding Plan

I intend to receive the grant personally. The requested USD 10,000 will fund 125 hours of my time at USD 80 per hour. The budget assumes no other funding or contributors.

The dataset will be updated through GitHub actions, with snapshots on GitHub Releases in a separate repository. I intend to maintain the pipeline after the grant, and public scripts will let others reproduce or mirror it. I am not requesting funds for recurring hosting costs.

With partial funding, I would publish at least 50 indexes meeting the same quality criteria and evaluate five packages. This would still include the metadata features and offline support described above, plus safe infix renaming. I would defer the other reliability fixes and agree on the hours, funding, and dates with the ISC before accepting partial funding.

Success

Definition of Done

The project will be complete when I have published a tagged Arity release and met the following criteria:

Language-server reliability Tests show that Arity resolves shared documentation, reports undocumented parameters, and handles references and call hierarchy consistently for quoted function names. Renaming produces valid R syntax or Arity refuses the operation.

Package knowledge At least 100 versioned indexes are published, with a documented schema and working weekly updates. For a package to count toward the target, its index must include arguments, defaults, and help for at least 80% of its exported functions. Export lists must match R's results for the same versions. Sampled arguments, defaults, and help must match or be withheld, and missing fields will be reported.

Tests show completion details, hover information, and signature help for packages that are not installed locally, including packages outside the existing bundle. The same tests pass offline after prefetch. Given the same source, configuration, and cached metadata, name resolution and diagnostics agree between the editor and CLI.

Validation and adoption I have published workflows and before-and-after results for ten packages, including checks in VS Code and Neovim, along with editor and CI guides and

maintenance instructions. The release passes the existing workspace and corpus checks with no unexplained regressions.

Measuring Success

The report will compare diagnostics and response times before and after the project using the ten package workflows. I will classify sampled diagnostic findings as defects, false positives, or unresolved cases, and report remaining limitations and any new regressions. For metadata, I will report coverage, accuracy, how current the data is, and extraction failures. The report will document the versions, configuration, and measurement setup so others can reproduce the comparison.

Feedback on the prerelease may add to these results, but completion will not depend on download counts or volunteer participation.

Future Work

Future work could cover more packages, select exact versions from a project's lockfile, support the CRAN Archive with package metadata from [ALLPACKAGES](#), and use runtime tracing. These are outside the scope of this grant.